



FPGA

Introduction

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Sommaire

1 Introduction

2 Building blocks of digital electronics

- Logic Variables and Functions
- Logic gates
- Why digital?
- A bit of history

3 VHDL: A Hardware description language

- Levels of Abstraction
- VHDL
- VHDL Basics
- Modelsim Tutorial

4 Multiplexers

- The 2-to-1 Multiplexer
- VHDL

5 Conclusion

Introduction

What is FPGA ?

- FPGA = Field-Programmable Gate Array
- Digital Integrated Circuit
- Difference is that FPGA can be *reprogrammed* = its hardware itself can be modified
- Example of digital circuit ?

General Modalities

Schedule

- 4 lecture sessions (8h)
- 3 tutorial sessions (6h)
- 4 lab sessions (16h)

Computer Science? Digital electronics? Microprocessors?

Module Informatique

- Computer Science
- Digital Electronics
- Microprocessors

Attention

- Interconnected disciplines
- Different issues
- Take a step back

Intended Learning Outcomes (ILO)

- Divided into two parts: Academic prerequisites; Professional skills

Prerequisites

- Computer Science
 - Boolean algebra, number representation
 - Algorithms
- Microprocessors
 - Operation of basic components (gate, flip-flop, mux, etc.)
 - Reading logic diagrams
 - Introduction to computer architecture

Intended Learning Outcomes (ILO)

Professional skills

- Design methodology (Top-down vs Bottom-up)
- Technologies (ASIC/FPGA/CPLD/GPU/ μ P), parallelism
- Hardware description languages (VHDL, Verilog)
- Metrics (complexity, power consumption, throughput/latency)
- Timings
- CAD tools (Vivado, Quartus)



Building blocks of digital electronics

Building blocks of digital electronics

Combinational logic

- Logic gates (AND, OR, NOT, NAND, NOR, XOR, XNOR)
- Multiplexers
- Decoders/Encoders
- Adders/Subtractors
- Comparators

Sequential logic

- Flip-flops (D, T, JK, SR) / Latches
- Registers
- Counters
- State machines

Logic Variables and Functions

Logic Variable and Function

Logic Variable (or binary, Boolean)

- Variable that can take two values
 - TRUE or FALSE
 - 1 or 0
 - H or L

Logic Function

- Result of the combination of one or more logic variables
- Combined together by Boolean mathematical operations and rules
 - Result depends on the input variables, can only be 0 or 1
 - A logic function has one or more logic input variables and one logic output variable
- This is called combinational logic

Representation of a Function

■ Truth table

a	b	c	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

- A function with 3 inputs has 8 combinations

■ Canonical form

$$Q = 1 \quad \begin{array}{l} \text{if } a = 0 \quad \text{and } b = 1 \quad \text{and } c = 0 \\ \text{or } a = 0 \quad \text{and } b = 1 \quad \text{and } c = 1 \\ \text{or } a = 1 \quad \text{and } b = 0 \quad \text{and } c = 0 \\ \text{or } a = 1 \quad \text{and } b = 0 \quad \text{and } c = 1 \end{array}$$

■ Algebraic form

$$Q = \bar{a} \cdot b \cdot \bar{c} + \bar{a} \cdot b \cdot c + a \cdot \bar{b} \cdot \bar{c} + a \cdot \bar{b} \cdot c$$

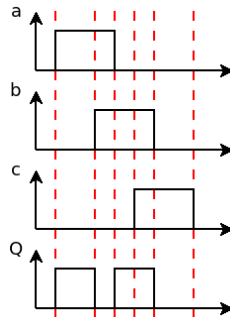
- **Minterm**: group of variables linked by AND
- Union (OR) of minterms

Representation of a Function

■ Truth table

a	b	c	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

■ Timing diagram

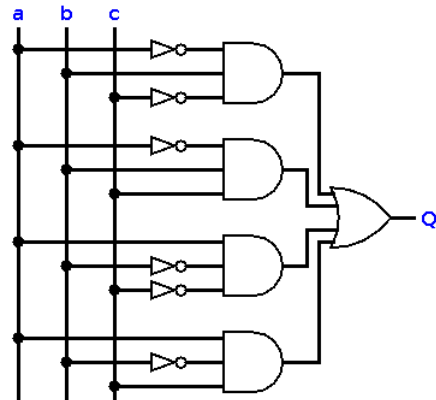


Representation of a Function

■ Truth table

a	b	c	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

■ Logic diagram



Logic gates

NOT gate

- Also known as:

- NOT function
- Inversion function
- Complement function

- $\bar{a}$ is the **complementary** value of a

- Truth table

a	Q
0	1
1	0

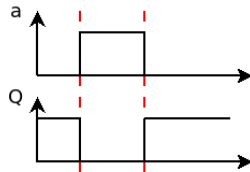
- Symbol



- Canonical form

$$Q = \bar{a}$$

- Timing diagram



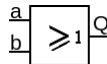
OR gate

- Also known as
 - OR function
 - Disjunction function
- Takes the value 1 when one **or** the other of the inputs is 1.
 - (or both)

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	1

■ Symbol



■ Canonical form

$$Q = a + b$$

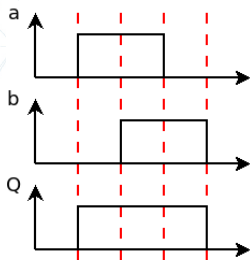
$$Q = a \vee b$$

OR gate

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	1

■ Timing diagram



■ Special properties

- $a + 1 = 1$
- $a + 0 = a$
- $a + a = a$
- $a + \bar{a} = 1$

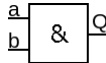
AND gate

- Also known as
 - AND function
 - Conjunction function
- Takes the value 1 when both inputs are 1.

■ Truth table

a	b	Q
0	0	0
0	1	0
1	0	0
1	1	1

■ Symbol



■ Canonical form

$$Q = a \cdot b$$

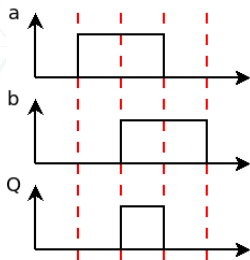
$$Q = a \wedge b$$

AND gate

■ Truth table

a	b	Q
0	0	0
0	1	0
1	0	0
1	1	1

■ Timing diagram



■ Special properties

■ $a \cdot 1 = a$

■ $a \cdot 0 = 0$

■ $a \cdot a = a$

■ $a \cdot \bar{a} = 0$

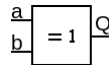
Exclusive OR gate

- Also known as
 - XOR function
- Takes the value 1 when one **or** the other of the inputs is 1.
 - Exclusively!

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	0

■ Symbol



■ Canonical form

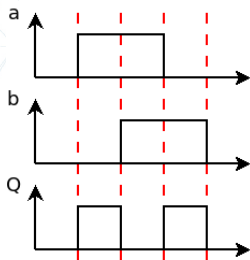
$$Q = a \oplus b$$

Exclusive OR gate

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	0

■ Timing diagram



■ Special properties

- $a \oplus 1 = \bar{a}$
- $a \oplus 0 = a$
- $a \oplus a = 0$
- $a \oplus \bar{a} = 1$

Exclusive OR gate

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	0

$Q = 1$ if $a = 1$ and $b = 0$
or $a = 0$ and $b = 1$

■ In other words

$$Q = a \oplus b = a \cdot \bar{b} + \bar{a} \cdot b$$

Remark

■ It can be built from other logic functions

■ Logic diagram

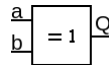
Exclusive OR gate

- Also known as
 - XOR function
- Takes the value 1 when one **or** the other of the inputs is 1.
 - Exclusively!

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	0

■ Symbol



■ Canonical form

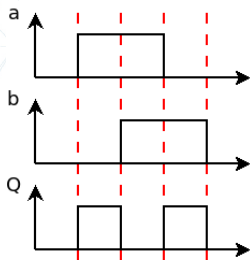
$$Q = a \oplus b$$

Exclusive OR gate

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	0

■ Timing diagram



■ Special properties

$$a \oplus 1 = \bar{a}$$

$$a \oplus 0 = a$$

$$a \oplus a = 0$$

$$a \oplus \bar{a} = 1$$

Exclusive OR gate

■ Truth table

a	b	Q
0	0	0
0	1	1
1	0	1
1	1	0

$Q = 1$ if $a = 1$ and $b = 0$
or $a = 0$ and $b = 1$

■ In other words

$$Q = a \oplus b = a \cdot \bar{b} + \bar{a} \cdot b$$

Remark

■ It can be built from other logic functions

■ Logic diagram



Why digital?

Did you say digital?

Analog vs Digital

- The states (inputs, internals, outputs) span a finite set of possibilities
- Deterministic, stable, and reliable transitions
- Synchronous
- Opposed to Analog Electronics, which operates on physical quantities (voltage, current...) in a continuous manner
- Necessity of conversion.

Digital Electronic Circuits

Some examples

- Microprocessor, DSP, microcontroller
- (GP)GPU (graphics card)
- Hardware accelerators
- System on Chip (SoC)
- FPGA/FPGA-SoC
- Dedicated circuit, ASICs, ...

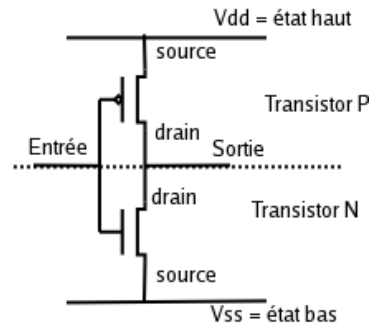
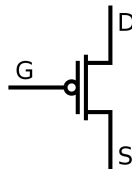


<http://youtu.be/9iXTAKv3pns>

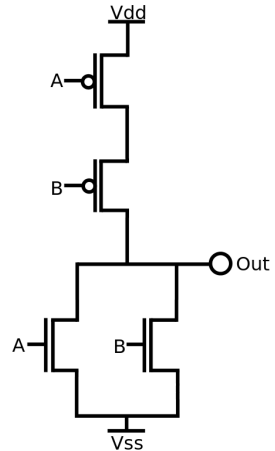
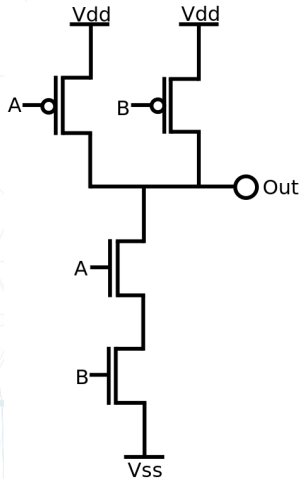
<http://youtu.be/CwhiuqNeBSg>

But why only two states?

- Implementation with switches (Open, Closed)
- Mechanical relay, then Vacuum tube, and finally Transistor
- CMOS technology
 - Combination of N-MOS and P-MOS

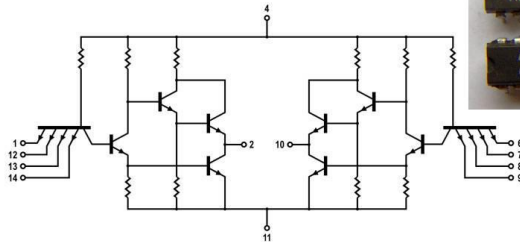
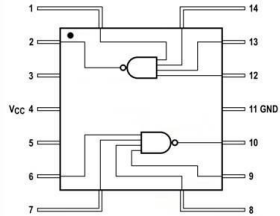
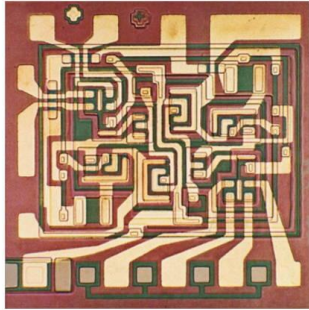


CMOS Technology

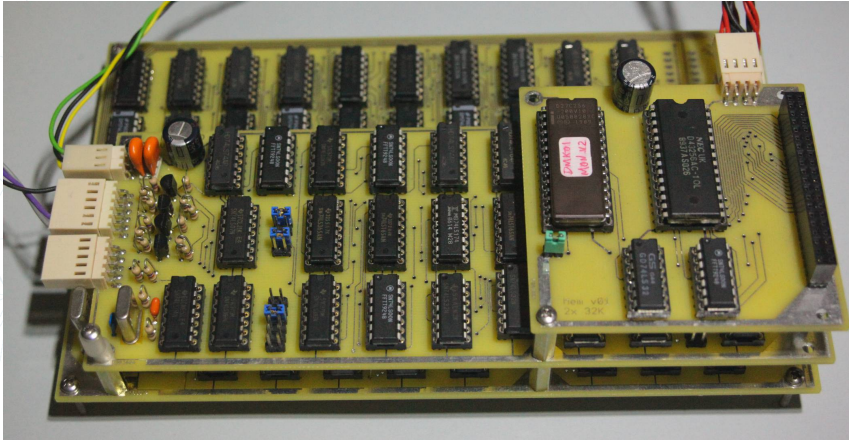


A bit of history

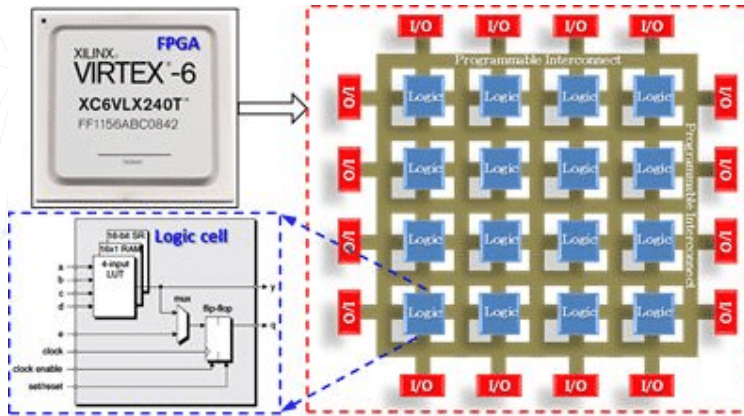
A long time ago...



... in a galaxy far, far away



Nowadays...



... we use VHDL on FPGA

```

-- Horizontal control signals
process(i_clk, i_rst_n)
begin
    if (i_rst_n = '0') then
        r_h_count    <= 0;
        o_hdmi_hs    <= '1';
        r_h_active    <= '0';
    elsif rising_edge(i_clk) then
        if (r_h_count = h_total) then
            r_h_count <= 0;
        else
            r_h_count <= r_h_count + 1;
        end if;

        if ((r_h_count >= h_sync) and (r_h_count /= h_total)) then
            o_hdmi_hs <= '1';
        else
            o_hdmi_hs <= '0';
        end if;

        if (r_h_count = h_start) then
            r_h_active <= '1';
        elsif (r_h_count = h_end) then
            r_h_active <= '0';
        end if;
    end if;
end process;

```

VHDL: A Hardware description language

The VHDL Language

- Hardware description language $\neq$ Programming language!
- Strongly typed, quite strict, very portable
- Can be used to describe hardware
- Can be used to describe behavior
- Can also be used to describe simulations

VHDL is a **synthesizable** language

Levels of Abstraction

Low-level

- Physical level (transistor / layout)
 - Abstraction: none – description of the circuit in terms of transistors, silicon layers, masks.
 - Languages: GDSII, SPICE, proprietary formats (Cadence, Synopsys).
 - Used for: ASIC design, foundry manufacturing.
- Gate-level logic
 - Abstraction: combinational and sequential logic circuits, based on logic gates.
 - Languages: generated netlists (e.g.: assign, flip-flops, mux, etc.).
 - Used for: post-synthesis simulation, precise timing.

Mid-level

■ RTL (Register Transfer Level)

- Abstraction: cycle-by-cycle description, data flow between registers via logical and arithmetic operators.
- Languages: VHDL, Verilog, SystemVerilog (synthesizable part).
- Used for: FPGA or ASIC synthesis, functional verification.

- This is the most common level for FPGA designers.

■ TLM (Transaction-Level Modeling)

- Abstraction: interactions between components modeled by transactions (e.g.: memory read/write), no fine timing details.
- Languages: SystemC (with TLM-2.0), sometimes Chisel or cocotb at the symbolic level.
- Used for: fast simulation, platform modeling, software prototyping.

- Allows much faster simulation than RTL
- Useful for testing software on a modeled SoC.

High-level

- Behavioral / algorithmic level

- Abstraction: the system does this or that, without specifying how (nor how it is synchronized).
- Languages: C/C++, MATLAB, Simulink, Python, etc.
- Used for: initial modeling, architecture exploration, automatic HDL generation (e.g.: via HLS).

- This level is often used as input for High-Level Synthesis (HLS) tools.

- System-level

- Abstraction: interconnection of complete subsystems (CPU, GPU, DMA, etc.), interaction with software, network, sensors. . .
- Languages: SystemC TLM, Simulink, or even functional description in Python.
- Used for: SoC design, architecture exploration, functional software verification (pre-hardware).

Summary (from lowest to highest)

Level	Example tool / language	Main usage
Physical	GDSII, SPICE	ASIC manufacturing
Gate-level	Netlist, EDIF	Post-synthesis simulation
RTL	VHDL, Verilog	Synthesizable design (FPGA/ASIC)
TLM	SystemC (TLM-2.0)	Fast system/performance modeling
Behavioral	C++, Python, HLS	Architecture exploration
System	SystemC, QEMU, Simulink	Full system simulation (HW+SW)

VHDL

Historical Context

- VHDL developed in the 1980s for the United States Department of Defense (DoD)
- VHSIC program (Very High Speed Integrated Circuit).
- Strongly influenced by the Ada language, standardized in 1983
- Ada = very structured, strongly typed language, focused on reliability, also designed for the DoD.

VHDL / Ada Similarities

- Strict typing
- Block scope (begin / end)
- Separate declarations (types, entities)
- Verbose and rigorous syntax
- Designed for critical systems
- Pascal-like heritage

Main Competitor: Verilog

■ Verilog vs VHDL: origins and style

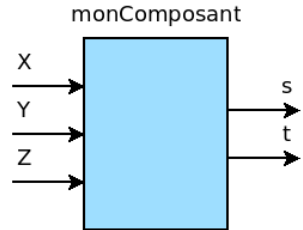
Aspect	VHDL	Verilog
Inspiration	Ada / Pascal	C / Perl
Origin	Military, DoD (USA, 1980s)	Gateway Design Automation (USA, 1984)
Target	Critical systems, safety	EDA industry, fast simulation
Style	Very formal, strongly typed	Light, concise, C-like
Typing	Strict (conversions needed)	Flexible (implicit typing)
Verbosity	Verbose, declarative	Concise, imperative
Readability	Robust but verbose	Fast to write, sometimes cryptic
Popularity	Widely used in Europe, academia	Widely used in USA, industry
Execution	"What it is" – structure declaration	"What it does" – behavior

VHDL Basics

VHDL Description of a Logic Component (1/2)

```
library ieee;
use ieee.std_logic_1164.all;

-- Entity = External view of the module
-- -----
entity myComponent is
  port (
    X : in std_logic;
    Y : in std_logic;
    Z : in std_logic;
    s : out std_logic;
    t : out std_logic
  );
end myComponent;
```



VHDL Description of a Logic Component (2/2)

```
-- Architecture = Internal view of the component (structural)
```

```
-----  
architecture myArch of myComponent is  
begin  
    s <= (X and Z) or (Y and not (Z));  
    t <= not( (X and Y) or Z);  
end myArch;
```

Summary

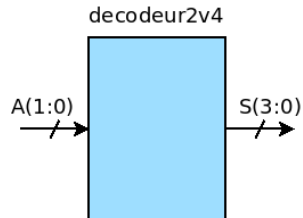
- **entity**: Describes the external interface of the component
- **architecture**: Describes the internal behavior of the component
- **architecture** is attached to an **entity**
- An **entity** can have several **architecture**

Example: 2-to-4 Decoder

```
entity decoder2to4 is
  port (
    A : in std_logic_vector (1 downto 0);
    S : out std_logic_vector (3 downto 0)
  );
end decoder2to4;

-- Functional description
-----

architecture arch of decoder2to4 is
begin
  S <= "0001" when A="00" else
       "0010" when A="01" else
       "0100" when A="10" else
       "1000";
end arch;
```



Modelsim Tutorial

Installing Modelsim

- VHDL is learned through practice
 - *TD machine*
 - With a simulator
- Install Modelsim version 20.1.1, available at the following address:

<https://www.intel.com/content/www/us/en/software-kit/750666/modelsim-intel-fpgas-standard-edition-software-version-20-1-1.html>

- This is the last version usable without a license :(
- Plan for 5 GB
- The Modelsim editor (and Quartus editor) are bad. You can install something else, like VSCode. In that case, also install an extension for VHDL.

decoder2to4.vhd

- Create a new folder
tuto_modelsim
- No spaces, no accents, no
Drive (prefer git)
- Launch VSCode (or another
editor)
- Copy the code on the right
- (Be nice to your prof: respect
indentation!)

```
entity decoder2to4 is
    port (
        A : in std_logic_vector (1 downto 0);
        S : out std_logic_vector (3 downto 0)
    );
end decoder2to4;

architecture arch of decoder2to4 is
begin
    S <= "0001" when A="00" else
        "0010" when A="01" else
        "0100" when A="10" else
        "1000";
end arch;
```

Why Simulate?

- Debugging C code on a microcontroller is relatively easy:
 - You run in step-by-step mode
 - And you find the error
 - (Soon, you won't be able to call the prof!)
- On FPGA, it's **much** more complicated
 - Not a program → Electronic circuit
 - Not a sequence of instructions → Interconnected LUTs + registers
 - Different architectures → Different timing
 - You can visualize some signals → But it's tedious
- In short: It's better to simulate
 - It's faster
 - Infinite number of "*probes*", bus visualization, etc...
 - Read/write to files
 - Test automation
- You spend **much** more time on the simulator than on the FPGA

decoder2to4_tb.vhd

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity decoder2to4 is  
end entity decoder2to4;
```

```
architecture tb of decoder2to4 is  
    signal tb_A : std_logic_vector (1 downto 0);  
    signal tb_S : std_logic_vector (3 downto 0);  
begin  
    uut: entity work.decoder2to4  
    port map  
    (  
        A => tb_A,  
        S => tb_S  
    );
```

```
process  
begin  
    tb_A <= "00";  
    wait for 10 ns;  
    tb_A <= "01";  
    wait for 10 ns;  
    tb_A <= "10";  
    wait for 10 ns;  
    tb_A <= "11";  
    wait for 10 ns;  
    tb_S <= "11";  
    wait for 10 ns;  
    wait;  
end process;  
end architecture tb;
```

Simulation script: decoder2to4.do

```
quit -sim

vlib work

vcom decodeur2v4.vhd
vcom decodeur2v4_tb.vhd

vsim -c work.decodeur2v4_tb

# INPUTS
add wave -divider Inputs:
add wave -color yellow uut/A

# OUTPUTS
add wave -divider Outputs:
add wave uut/S

run -all
```

Simulation

- Launch Modelsim
- In the Modelsim command prompt, navigate to your working directory using the `cd` command

```
cd path/to/project
```

- You can check that the files are present with the `ls` command

```
ls
```

- Simulate using the command below:

```
do decoder2to4.do
```



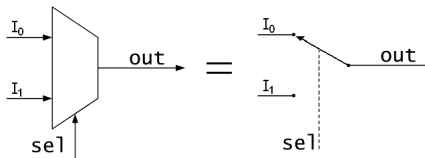
Multiplexers

The 2-to-1 Multiplexer

The 2-to-1 Multiplexer

The multiplexer

- Sometimes abbreviated as MUX, allows selection of one output among N inputs.
- A selection bit allows choosing between 2 inputs.



<i>sel</i>	I_1	I_0	<i>out</i>
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

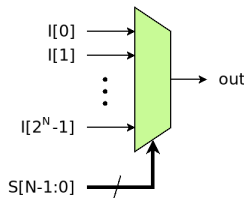
- Simply

<i>sel</i>	<i>out</i>
0	I_0
1	I_1

$$out = \overline{sel} \cdot I_0 + sel \cdot I_1$$

Multiplexer with Multiple Inputs

- 1 selection bit
 - 2 inputs
- 2 selection bits
 - 4 inputs
- N selection bits
 - 2^N inputs
- In other words:
 - $\log_2(M)$ selection bits for M inputs



s_{N-1}	$\dots$	s_1	s_0	out
0	$\dots$	0	0	I_0
0	$\dots$	0	1	I_1
$\dots$				$\dots$
1	$\dots$	1	1	I_{2^N-1}

VHDL Implementation: Different Methods

- `with ... select`
- `when ... else`
- `if ... then ... else` in a combinational `process`

VHDL

Description of a 4-to-1 Multiplexer

```
entity mux4to1 is
  port (
    A : in std_logic;
    B : in std_logic;
    C : in std_logic;
    D : in std_logic;

    S : in std_logic_vector (1 downto 0);

    Q : out std_logic
  );
end mux4to1;
```

Structure with ... select

```
architecture with_select of mux4to1 is
begin
    with S select
        Q <= A when "00",
              B when "01",
              C when "10",
              D when "11",
              'U' when others;
end architecture with_select;
```

Structure when ... else

```
architecture when_else of mux4to1 is
begin
    Q <= A when S = "00" else
        B when S = "01" else
        C when S = "10" else
        D when S = "11" else
        'U';
end architecture when_else;
```

Structure case ... when

```
architecture case_when of mux4to1 is
begin
    process(S, A, B, C, D)
    begin
        case S is
            when "00" =>
                Q <= A;
            when "01" =>
                Q <= B;
            when "10" =>
                Q <= C;
            when "11" =>
                Q <= D;
            when others =>
                Q <= 'U';
        end case;
    end process;
end architecture case_when;
```

Structure if ... then ... else

```
architecture if_then of mux4to1 is
begin
    process(S, A, B, C, D)
    begin
        if (S = "00") then
            Q <= A;
        elsif (S = "01") then
            Q <= B;
        elsif (S = "10") then
            Q <= C;
        elsif (S = "11") then
            Q <= D;
        else
            Q <= 'U';
        end if;
    end process;
end architecture if_then;
```

Conclusion

Conclusion

At the end of this course, I am able to

- List the basic components used in digital electronics
- Explain how a multiplexer works
- Describe a 4-to-1 multiplexer in VHDL
- Simulate a VHDL component with Modelsim